Mini: Project documentation
Stage 04

Karel Šafr

1 Your project: stage 04

You will implement the declaration protocol and block traversal in the resolver: allocate a distinct identity, reject duplicates, check the initializer before permitting reads, then enter and leave each nested scope. The parser, AST and C++ backend are ready. Lookup still searches only the current map; searching enclosing scopes comes next.

Use the start version for your own work. The table below shows what already works and what your change will add. Follow the exercise in order, then compare your results with the solution when it is provided.

Area	**Start**	**After completion**
Lexing and parsing	Names, declarations, assignments and blocks are recognized.	Unchanged.
Bare literal execution	Works end to end; no declaration or nested block is needed.	Unchanged.
Declarations	VarStatement, declare and define contain explicit TODO bodies.	A distinct undefined binding is inserted, its initializer checked, then it becomes defined.
Nested blocks	AST and lower work; block resolution reaches a TODO.	One fresh map is entered, visited and removed for each block.
Lookup	The supplied method searches only the current map.	Still current-only; an outer-only name inside a block is rejected.
Name identity	AST nodes have optional SymbolId fields.	Each declaration receives one fresh ID; supplied use visitors reuse it.
Excluded features	Operators, branches, loops, parameters, general calls and returns.	Still excluded.

How to use this document

Begin with ASSIGNMENT.txt in this folder. Run the starting example, read the task, and then use this guide to trace the relevant code. The file reference explains all 31 source and configuration files. When I provide the solution, its READ_ME.txt will help you compare your approach with the worked solution.

You have everything you need for this exercise in the folder you received: the compiler source, runtime, build configuration, examples and checks. Our compiler translates Mini to C++17 and then uses a C++ compiler to produce a native executable. Follow these separate steps when you explain how a Mini program runs.

2 Build the compiler and make a first input

Requirements: CMake 3.20 or newer, a C++17 compiler and its standard library. The commands below use a macOS shell and Unix Makefiles. Run them in a writable working copy of the selected package, in the directory containing CMakeLists.txt. The project does not download third-party C++ libraries.

```
cmake --version
c++ --version
cmake -S . -B build -G "Unix Makefiles" \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
./build/minic --help
```

The supplied inputs/ directory contains starting_point.mini, acceptance.mini and edge cases. Save this additional manual demonstration as inputs/hello.mini:

```
fun main() {
  print 42;
  print "Mini";
}
```

```
./build/minic tokens inputs/hello.mini
./build/minic ast inputs/hello.mini
./build/minic run inputs/hello.mini
```

The run command prints 42 and Mini on separate lines and exits 0 in both the untouched start and completed solution. This demonstration avoids the exercise TODOs.

Run the supplied baseline and checkpoint checks

```
./build/minic run inputs/starting_point.mini
python3 CHECK_PROJECT.py --mode start --compiler ./build/minic
# After editing and rebuilding:
python3 CHECK_PROJECT.py --mode solution --compiler ./build/minic
```

The baseline works before editing. Python 3.8+ is needed only for the optional checker. PASS includes matched expected errors; start-mode PASS does not mean completion. Rebuild after C++ edits, using a separate build for each stage. Later examples abbreviate ./build/minic as minic; substitute your executable path. ASSIGNMENT.txt lists the bundled inputs; other manual examples may need to be saved separately.

3 Architecture: calls, data and two builds

Order / data	Responsible component
1. SourceText	source.cpp
2. tokens	lexer.cpp
3. parsed AST	parser.cpp
4. core AST	lower.cpp
5. resolved AST	resolver.cpp
6. C++ text	codegen.cpp
7. native program	HostCompiler

driver.cpp coordinates the path; build/run add the external C++ compiler.

main.cpp is the process boundary; cli.cpp validates arguments. SourceText::fromFile reads bytes before executeCommand drives the shared pipeline. A command selects how far to proceed; it does not select an independent parser.

The two compilations are different

- CMake builds our C++ implementation into the mini_compiler static library and the minic executable. configure_file generates mini_build_config.hpp in the build directory.
- minic cpp emits your Mini program as C++ text. minic build/run invokes HostCompiler to compile that generated translation unit with the header-only runtime. The resulting program is a different executable.
- The runtime executes Value operations and printing when that generated executable runs. It is not the lexer, parser or resolver, and it is not linked as a separate runtime library.

Current tree and pass contracts

Program.main_body holds the statements in the single main function. Resolution is an empty result type. Name bindings are stored as SymbolId annotations in AST nodes, not in the empty result object.

For this stage, lowering is an identity pass. It is visible in the driver so the same pipeline can grow without changing the CLI.

SourceText owns the source string. Tokens own lexeme/payload data and retain source spans. AST variants encode node kinds; unique_ptr owns recursive children. Moves transfer ownership between parser/lowering results. This compiler memory is distinct from scope tables and from values in a running Mini program.

4 Commands, output files and errors

Command	Last phase / observable result
tokens FILE	scan; prints the token sequence including EOF. A fragment may be enough.
ast FILE	parse; prints the parsed tree before lowering and resolution.
lower FILE	lower; prints the core tree before resolution.
check FILE	resolve; success is silent, status 0. No C++ is emitted or executed.
cpp FILE [-o OUT.cpp]	emitCpp; writes text to stdout or to the given file.
build FILE [-o OUT]	host C++ compilation; creates an executable but does not run it.
run FILE	host C++ compilation then execution; uses and cleans a temporary directory.
--help	CLI only; no input is read. For the available features, use the opening scope table.

Without -o, build chooses an executable beside the input (normally the stem, with the platform suffix). Choose an explicit output name when you want to keep the generated file. cpp/build reject overwriting the Mini input. run forwards the generated program output and its exit status; it does not leave a reusable executable.

```
./build/minic cpp inputs/hello.mini -o build/hello.cpp
./build/minic build inputs/hello.mini -o build/hello
./build/hello
```

Status / phase	Interpretation
0	Requested operation succeeded; check/build can legitimately print nothing.
1 / lex, parse, resolve, codegen, io, backend	Controlled compiler error; inspect its phase and location where provided.
1 / runtime	Generated program failed during an operation; e.g. incompatible operands.
2 / cli	Wrong command/arguments. No-argument minic is not the help command.
70 / internal	Unexpected std::exception. The stage08 lowering TODO deliberately uses this path.

check is not a dynamic type checker and cannot prove that execution will succeed. A parsed program can still fail resolution; a resolved program can still fail code generation at a TODO or fail later at runtime. Follow the earliest failing phase.

5 Starting contracts: declarations, identities and scope maps

Your project recognizes variables, assignments and blocks. Their parser rules, AST nodes, formatting and C++ generation are supplied. You complete declare(), define(), and the VarStatement and BlockStatement branch bodies in src/resolver.cpp. Together these parts establish which declaration a name refers to and when that declaration becomes usable. No new parser rules or runtime storage are required.

Start with a bare literal print. Resolver::run() already opens the initial scope for main, visits its statements and closes that scope. Literals need no binding, so this complete native program runs

before the edit. A variable declaration instead reaches the declaration-visitor TODO; a nested block reaches its own TODO. tokens, ast and lower stop before resolution and can already show both forms.

Concrete contracts used by the resolver

Type or operation	Role in this package
Program.main_body	Ordered statements of the one supported main function.
BlockStatement.statements	Ordered children visited inside one fresh scope.
SymbolId / next_symbol_	A declaration identity and the counter used to allocate the next one.
Binding {id, defined}	Compiler metadata: which declaration, and whether its initializer has been checked.
scopes_	A vector of name-to-Binding maps; its last map is current.
resolve(source, program)	Annotates AST nodes in place and returns an empty Resolution object.

Keep resolveName() unchanged: it searches only scopes_.back(). Once you complete declaration handling, a read may use a defined binding in the current map. A declaration in a new block may shadow an outer name, but an outer-only name is rejected from that block for now. This temporary rule separates managing scope boundaries from the next exercise on outward search.

Each successful declaration gets a distinct ID, including a same-spelled declaration in a sibling block. Popping a scope removes names from lookup; it does not reset the ID counter. The supplied generator uses the IDs to choose separate C++ variable names. During compilation the maps contain bindings and flags, not the changing values of your running program.

Lowering remains identity because the grammar needs no rewrite. The AST owns compiler nodes, while generated C++ owns runtime storage. Neither AST ownership pointers nor scope maps are a runtime memory manager. Keep syntax structure, declaration identity and execution values separate when explaining an example.

6 Implement one declaration-and-scope protocol

Keep the existing method signatures and dispatch conditions. Replace the four TODO bodies in src/resolver.cpp: declare(), define(), the VarStatement branch and the BlockStatement branch. Remove their placeholder throws and warning-suppression (void) expressions. Other resolver visitors, current-only lookup, parser, AST, generator and runtime remain unchanged.

Allocate and define one declaration

- In declare(), take the current map and copy next_symbol_ into a local ID. Attempt to insert the source name with Binding{id, false}. Check the insertion result rather than overwriting an existing binding.
- If insertion fails, throw a resolve CompileError at variable.name_span with "duplicate variable '<name>' in the same scope". Check only this map; the same spelling in an outer map is allowed.
- After successful insertion, record the ID in variable.resolved_id and increment next_symbol_ once. Do not allocate IDs for reads or reset the counter at block entry.
- In define(), find the already inserted name in the current map and set defined to true. Do not create another binding or change its identity.

Connect declaration handling to traversal

```
declare the variable as {new id, defined = false}
resolve its initializer expression
define that same current-scope name
return from the handled statement branch
```

This is explanatory pseudocode for your VarStatement branch. The order matters: the new name is already present while its initializer is checked, so the supplied lookup rejects a self-read. Only successful initializer resolution reaches define(). Defining early would hide that error; defining never would reject all later reads. The final return avoids the unsupported-statement diagnostic.

Give each block one map

The existing mutable block node exposes its statements vector. Call beginScope(), resolve Statements(block->statements), endScope(), then return. Enter before any child is visited; ordered traversal recursively visits nested blocks. One map per statement would hide duplicates and break later reads. Reusing the parent map would confuse valid shadowing with a same-scope duplicate.

Every successfully traversed block pairs one entry and one exit. If a child throws CompileError, compilation stops and its resolver is discarded; recovery is outside this task. The generator already emits braces and distinct C++ variables from your annotations. Your work establishes binding correctness before that backend runs.

7 Worked shadowing example: two declarations, two bindings

Save the source below as w04_blocks.mini in your project folder. Before editing, tokens and ast succeed, and lower prints the same tree. check and run stop at the outer declaration with "resolve: TODO WEEK04: resolve variable declaration", exit 1 and empty stdout. The whole program must resolve before generated code executes; no earlier print could run first.

```
fun main() {
  var x = 10;
  { var x = 20; print x; }
  print x;
}
```

After implementing all four bodies and rebuilding, use minic ast w04_blocks.mini, minic check w04_blocks.mini and minic run w04_blocks.mini. The complete AST stdout is unchanged:

```
(program
  (fun main ()
    (var x (number 10))
    (block
      (var x (number 20))
      (print (variable x))
    )
    (print (variable x))
  )
)
```

Hand trace of binding decisions, not a CLI dump

Point in traversal	Decision
Outer var x	Allocate SymbolId 0; check literal 10; mark defined.
Enter block; inner var x	Create a new map; allocate SymbolId 1 for its local x.
Inner print	Find x in the current inner map: use ID 1.
Leave block; final print	Pop the inner map; find x in main: use ID 0.

check now returns 0 without output. run returns 0 and prints 20 followed by 10, each on its own line. The values are stored only when the generated program executes. During resolution, the scope maps contain identities and flags; they do not contain the numbers 10 and 20 as variable values.

For an additional observation, minic cpp w04_blocks.mini shows distinct generated names mini_variable_0 and mini_variable_1 inside corresponding C++ braces. This is how the already supplied backend keeps the two declarations separate. The second declaration never assigns to the first. Leaving the block changes lookup visibility; it does not restore a previously overwritten outer value.

The AST formatter prints source names, not resolved_id fields, so the ast text looks the same before and after resolution. Use the hand trace to explain the IDs, then compare it with the generated variable names and the run result. These observations show different parts of the same process: syntax, binding and execution.

8 Acceptance checks, the current-only limit and stage 05

Baseline: a bare literal needs no binding

```
fun main() { print 10; }
```

Save this source as w04_local.mini in your project folder. Before and after the edit, minic check w04_local.mini succeeds silently, and run prints 10 followed by a newline. The bundled inputs/starting_point.mini contains the same source. It uses main's supplied traversal and literal print, while the declaration and nested-block bodies remain unfinished.

Boundary: an outer-only variable is still unavailable inside a block

```
fun main() { var x = 10; { x = 30; } print x; }
```

Save this source as w04_outer.mini. The untouched start reaches the declaration-visitor TODO first. After completion, minic check w04_outer.mini still exits 1, now with "resolve: unknown variable 'x' in current scope". The empty inner map has no x and lookup does not continue to main. Keep outward search for the next exercise.

Run the block-local y example in ASSIGNMENT.txt: declare y in a nested block, then print y after that block. The later use must be rejected. When the block ends, its map is removed and y is no longer visible. Compare this with the outer-only example: searching an enclosing scope and recovering a finished child scope are different operations.

What you should be able to demonstrate

- Complete declare, define and the two statement bodies. Check distinct declaration IDs, duplicate rejection, initializer ordering and one map per block.

- The literal baseline remains valid; the worked example prints 20 then 10. The bundled edges check nested restoration, sibling reuse, leakage, local assignment and the deliberate outer-access rejection.
- Run minic cpp inputs/edge_distinct_ids.mini after completion: two declarations generate mini_variable_0 and mini_variable_1. Explain why IDs are distinct even though maps store no runtime values.

Keep your completed declaration and block protocol. Stage 05 retains these parts and introduces TODOs for lookup, expression traversal and assignment binding. Even local reads and grouped literals will wait for that work, while a bare literal still runs. We will search from the current map outwards and attach existing declaration IDs to every read and assignment target.

9 File reference: responsibilities

Use this table to locate the code in your project folder. The two counts give each file's total length in start and solution, including comments and blank lines. They are not line numbers to edit. The worked solution adds 18 nonblank code lines, excluding comments and the unchanged skeleton. Your implementation may use a different number of lines; follow the task and explain how your code produces the required behavior.

File	Total lines: start / solution	Responsibility and boundary
.gitignore	7 / 7	Version-control exclusions for generated build/output files. It is configuration, not a compiler phase.
CMakeLists.txt	57 / 57	Requires CMake 3.20 and C++17; configures mini_build_config.hpp, builds mini_compiler and links minic. Run the bundled checkpoint checks separately.
runtime/mini_runtime.hpp	46 / 46	Header-only Value representation, printing and the operations available in this stage. Included by generated programs, not a replacement for the frontend.
src/ast.cpp	108 / 108	formatAst: readable structural dump shared by ast and lower. This is a view of the tree, not an evaluator or stable serialization format.
src/ast.hpp	97 / 97	Owns the stage-specific Program, Expression and Statement data structures. Tagged variants distinguish node kinds; unique_ptr owns recursive children.
src/build_config.hpp.in	15 / 15	CMake template for compiler path/family, runtime include directory and executable suffix. Its generated header belongs to the build directory.
src/cli.cpp	129 / 129	Validates command syntax and -o usage; formats common help. The final-language help paragraph does not promise all features in an early stage.
src/cli.hpp	37 / 37	CommandKind and CommandLine: command, input path and optional output path. Declares parse CommandLine and helpText.
src/codegen.cpp	197 / 197	Traverses the current tree, emits values/statements, and creates the host main wrapper. See the stage-specific sections for supported nodes and the exercise.
src/codegen.hpp	19 / 19	emitCpp(const Program&, const Resolution&) returns one C++17 translation unit. It consumes the resolved tree; it does not launch a compiler.

File	Total lines: start / solution	Responsibility and boundary
src/diagnostics.cpp	69 / 69	Maps diagnostic phases to labels and formats source-name/line/column when available. Supplies consistent compiler-side error messages.
src/diagnostics.hpp	49 / 49	Phase and CompileError contracts, optional source location and diagnostic formatting interface. Compile Error is distinct from RuntimeError.
src/driver.cpp	200 / 200	Runs phases in order and stops at the requested view. Writes C++, manages temporary directories and invokes HostCompiler for build/run.
src/driver.hpp	20 / 20	executeCommand(const CommandLine&, const Source Text&, ostream&) is the shared orchestration entry point used by the executable.
src/host_compiler.cpp	281 / 281	Selects and invokes the external C++ toolchain, quotes arguments, captures compilation logs and normalizes process status. Checks fresh executable output.
src/host_compiler.hpp	44 / 44	HostCompilerConfig and HostCompiler: configured toolchain, compile(source, executable, scratch), and run(executable).
src/lexer.cpp	246 / 246	Reads characters and creates tokens with spans and literal payloads, skipping whitespace/comments. Keyword recognition and grammar acceptance are separate.
src/lexer.hpp	19 / 19	scan(const SourceText&) returns vector<Token>. The public function hides the scanner cursor and token-building helpers.
src/lower.cpp	14 / 14	Provides the explicit lowering stage. Identity before stage08; from stage08 rewrites parsed for into core blocks and while after the exercise is completed.
src/lower.hpp	17 / 17	lower(Program) returns Program. Taking the tree by value supports moving its owned children instead of copying them.
src/main.cpp	41 / 41	Process entry: collects argv, parses CLI, loads Source Text and calls driver. Catches compiler errors and unexpected C++ exceptions at the boundary.
src/number_format.cpp	43 / 43	Implements locale-independent numeric formatting used by diagnostics/dumps and generated code; avoids decimal-comma differences.
src/number_format.hpp	17 / 17	Declares formatNumberForDisplay and formatNumber ForCpp. A human-readable number and a valid C++ double literal serve different purposes.
src/parser.cpp	229 / 229	Recursive-descent parser for the current stage. Constructs AST nodes and reports the first syntax error; does not execute expressions or resolve names.
src/parser.hpp	20 / 20	parse(const SourceText&, const vector<Token>&) returns Program. Source text supports diagnostics; tokens supply the grammar input.
src/resolver.cpp	142 / 145	Name-analysis pass: no-op in stage03, local/scope binding from stage04, outward lookup after stage05 and function resolution from stage09.
src/resolver.hpp	19 / 19	resolve(const SourceText&, Program&) returns Resolution and may annotate AST names. The exact result and lookup capabilities grow by stage.

File	Total lines: start / solution	Responsibility and boundary
src/source.cpp	77 / 77	Loads the input file, indexes line starts and maps offsets to source positions. Shared by lexical, syntax and semantic diagnostics.
src/source.hpp	44 / 44	SourceText owns source name, contents and line starts; SourceSpan stores offset, length, line and column. Declares file loading and span lookup.
src/token.cpp	119 / 119	formatTokens: prints token kinds, locations, lexemes and payloads. EndOfFile is explicit; token names describe scanning, not full language support.
src/token.hpp	66 / 66	TokenKind, TokenLiteral and Token. Each token owns lexeme text, a source span and an optional double/ string literal payload.

Use CHECK_PROJECT.py, checks/CASES.json and inputs/ for the checks described in ASSIGNMENT.txt. These support files are separate from the compiler sources listed here. A worked explanation accompanies the solution.

10 Working on your project

Keep the build and runtime together

Keep runtime/ beside src/. If you move the project, configure and build it again in its new location; the build records paths to the compiler and runtime. HostCompiler uses MINI_CXX if set, then CXX, then the compiler chosen by CMake. These overrides take one executable name or path, without extra flags.

Run the project in your IDE

In CLion, open the folder containing CMakeLists.txt and build the minic target. Set the program arguments to run inputs/starting_point.mini. Set the working directory to the project folder. Your IDE may use its own build directory; use the minic executable from that directory when following the command-line examples.

What you see	What to do
Cannot open input	Check the working directory and filename, including the .mini extension.
The old TODO still appears	Rebuild. Check that you are running minic from the copy you edited.
A backend error	Read the compiler log in the error. Check your C++ toolchain and the runtime path.
An unsupported feature	Check the opening scope table and the examples for this stage.
CTest finds no tests	Use CHECK_PROJECT.py or the manual commands in ASSIGNMENT.txt.

Before you finish

Your change in this exercise is in src/resolver.cpp. Rebuild, run the acceptance example and complete the edge cases in ASSIGNMENT.txt. The solution mode of CHECK_PROJECT.py checks your edited start as well as the worked solution.

- Show an example that works before your change, then explain the new behavior after your change.
- Point to the function you changed. Explain the order of its steps and why the edge cases matter.
- For a rejected example, identify the failing phase and explain why that error is expected.

Keep your completed work

Save this completed project separately. I will provide a new starting project for our next exercise. It will include the supporting changes needed for that topic. Open and build that new project as a whole; copying one file between versions can mix incompatible parts.